

Sentinel AI Security Audit Report

Audit Metadata

- **Filename:** insecure_code - Copy.c
- **SHA-256 Checksum:** 6c6254f02b59eabd85c927dacb844e1c6e4268aca4dadf0d5cc5c61f14954054
- **Verdict:** VULNERABLE
- **Risk Level:** CRITICAL
- **Total Functions Evaluated:** 1

Executive Summary

The Sentinel AI automated binary analysis engine has completed its assessment of the provided source artifact. The audit has returned a **VULNERABLE** verdict with a **CRITICAL** risk rating.

The analysis identified a severe memory safety violation within the authentication logic. Specifically, the function ``vulnerable_login`` exhibits classic symptoms of a stack-based buffer overflow. This vulnerability allows an attacker to provide input that exceeds the allocated memory boundaries of a local buffer, leading to the corruption of the function's return address. In a production environment, this flaw could be weaponized to achieve **Arbitrary Code Execution (ACE)** or a complete system compromise. Immediate remediation is required before this code is integrated into any deployment pipeline.

Analysis Metrics

- **Analysis Engine:** GATv2 (Graph Attention Network) Compiler Wrapper
- **Static Analysis Mode:** Disassembled Control Flow Graph (CFG) Analysis
- **Total Functions Scanned:** 1
- **Flagged Functions:** 1
- **Average Model Confidence:** 95.74%
- **Security Standards Mapped:** CWE-787, CWE-119, CWE-120, SEI CERT ARR30-C

Detailed Findings

Finding 1: Function ``vulnerable_login``

- **Model Confidence:** 95.74%
- **Identified Threat:** CWE-119 (Memory Safety / Buffer Overflow)

Technical Breakdown of Exploit Vector

Analysis of the disassembled Basic Block 0 reveals a critical stack management flaw. The function allocates ``0x30`` (48 bytes) for its stack frame (``SUB RSP, 0x30``). However, the local buffer is addressed at ``[RBP - 0x10]``, providing only 16 bytes of space before reaching the saved Base Pointer (RBP) and the Return Address.

The instruction sequence:

1. ``LEA RAX, [RBP - 0x10]`` (Loads the address of the 16-byte buffer)
2. ``CALL 0x00003e68`` (Invokes a data-copying function, likely ``strcpy`` or ``gets``)

The call at ``0x00003e68`` does not receive a length argument, indicating a lack of bounds checking. An input string longer than 16 bytes will overwrite the stack metadata. By carefully crafting this overflow, an attacker can redirect the instruction pointer (RIP) to a malicious payload or a Return-Oriented Programming (ROP) chain upon the execution of the ``RET`` instruction in Basic Block 1.

SEI CERT C Coding Rule Violated

- **ARR30-C:** Do not form or use out-of-bounds pointers or array subscripts. The code fails to validate that the source string length is within the bounds of the destination stack buffer.

Remediation & Secure Code Fix

To mitigate this vulnerability, the developer must replace unsafe memory copy operations with bounded alternatives and validate input lengths.

Vulnerable Concept (C):

```
void vulnerable_login(char *input) {
    char buffer[16];
    strcpy(buffer, input); // CRITICAL VULNERABILITY
}
```

Secure Refactored Code (C):

```
#include <string.h>
#include <stdio.h>

#define MAX_BUFFER_SIZE 16

int secure_login(const char *input) {
    char buffer[MAX_BUFFER_SIZE];

    if (input == NULL) return -1;

    // Use strncpy to enforce bounds and manually ensure null-termination
    strncpy(buffer, input, MAX_BUFFER_SIZE - 1);
    buffer[MAX_BUFFER_SIZE - 1] = '\0';

    // Alternatively, use safer C11 bounds-checking interfaces if available:
    // strcpy_s(buffer, sizeof(buffer), input);

    return 0;
}
```

General Mitigations

To provide a defense-in-depth posture, the following system-level mitigations and best practices should be implemented:

1. **Stack Canaries (Stack Smashing Protector):** Enable compiler flags (e.g., ``-fstack-protector-all``) to insert "canary" values before the return address. If a buffer overflow occurs, the canary is corrupted, and the program terminates before the hijacked return address can be used.
2. **Address Space Layout Randomization (ASLR):** Ensure the target environment has ASLR enabled to randomize the memory addresses of the stack, heap, and libraries, making ROP-based exploits significantly harder to execute.

3. **Data Execution Prevention (DEP / NX Bit):** Mark the stack as non-executable to prevent the execution of shellcode injected into buffers.
4. **Control Flow Integrity (CFI):** Utilize modern compiler features that validate the integrity of the program's control flow graph at runtime, preventing unauthorized jumps to unexpected code locations.
5. **Static & Dynamic Analysis (SAST/DAST):** Integrate automated tools like this GNN-based auditor into the CI/CD pipeline to catch memory safety issues during the development phase.

End of Report

Sentinel AI Security Division